CIS5200: Machine Learning Spring 2026

Neural Networks

Date: March 19, 2026

Over the last several lectures, we've been talking about how to make linear models more powerful. The basic idea is that, if a model $h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$ is not powerful enough, we can just "make up" new features using a *feature map* ϕ so that a model $h(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x})$ is more powerful – in the first model, there are d weights, and in the second model there are $D \gg d$ weights. More features means more power!

So far, we've only seen specific kinds of feature maps though: ones where we can find *kernel functions* $k(\mathbf{x}, \mathbf{x}')$ that compute inner products in the feature mapped space: $k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$. Then, for models where we only ever access data through inner products, we can swap in the nice, efficient kernel function everywhere the inner product appears and do learning tractably.

There are a couple problems with this idea that make it somewhat inflexible:

1. The kernel trick is great, but we can only choose feature maps $\phi(\mathbf{x})$ for which either (1) we know an efficient kernel function $k(\mathbf{x}, \mathbf{x}')$, or (2) we can afford to just directly compute $\phi(\mathbf{x})$.
2. Since using the kernel trick required the substitution $\mathbf{w} = \sum_{i=1}^n \alpha_i \phi(\mathbf{x}_i)$, making a prediction $h(\mathbf{z}) = \sum_{i=1}^n \alpha_i k(\mathbf{x}_i, \mathbf{z})$ requires n kernel computations, or $O(dn)$ time minimum.
3. A lot of the kernels we've seen don't seem very good for data modalities like images – e.g. the RBF kernel still computes Euclidean distances, which we know are bad for images (think about the example of moving a cat picture 1 pixel to the right).

The idea of neural networks is simple: let's still use a feature map like $\phi(\mathbf{x})$ (although we'll change the letter from ϕ in a minute), but let's *learn* the feature maps!

1 A Bad Idea

Let's start from the beginning. We know a number of ways to learn linear models of the form:

$$h(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$$

We can then use this model directly in the regression setting, or we can do something like $\text{sign}(h(\mathbf{x}))$ to do classification, and we've seen a variety of loss functions that encourage good performance either at regression or at classification. For kernel methods, or at least for the polynomial kernel, the basic idea of the feature map was to combine existing features to create new ones. One natural question is: why not just pick a function that combines features then?

An Almost Good Idea

Suppose we are given some training set $\mathbf{x}_1, ..., \mathbf{x}_n$ with labels $y_1, ..., y_n$. We think linear models of the form $h(\mathbf{z}) = \mathbf{w}^\top \mathbf{z}$ are too simple. Let's learn a feature map of the form:

$$\phi(\mathbf{x}) = \mathbf{V}\mathbf{x}$$

where $\mathbf{V} \in \mathbb{R}^{D \times d}$ is a matrix that just maps $\mathbf{x}$ to a higher, $D > d$ dimensional space by taking linear combinations of the existing features. Then, we'll use models of the form:

$$h(\mathbf{z}) = \mathbf{w}^\top \phi(\mathbf{z}) = \mathbf{w}^\top \mathbf{V}\mathbf{z}$$

Before getting to why the above doesn't work, let's examine what is actually going on. $\mathbf{V}$ is a matrix of the form:

$$\mathbf{V} = \begin{bmatrix} v_{11} & \cdots & v_{1d} \\ \vdots & \ddots & \vdots \\ v_{D1} & \cdots & v_{Dd} \end{bmatrix}$$

Here D is the dimensionality of the new space we're mapping $\mathbf{x} = [x_1, ..., x_d]$ to, and d is the dimensionality of the original input space. Computing $\mathbf{V}\mathbf{x}$ amounts to computing new features. Let's call the resulting vector of features $\mathbf{a} = \mathbf{V}\mathbf{x}$. It has the form:

$$\begin{aligned} \mathbf{a}_1 &= v_{11}x_1 + v_{12}x_2 + \cdots + v_{1d}x_d = \mathbf{v}_1^\top \mathbf{x} \\ \mathbf{a}_2 &= v_{21}x_1 + v_{22}x_2 + \cdots + v_{2d}x_d = \mathbf{v}_2^\top \mathbf{x} \\ &\vdots \\ \mathbf{a}_D &= v_{D1}x_1 + v_{D2}x_2 + \cdots + v_{Dd}x_d = \mathbf{v}_D^\top \mathbf{x} \end{aligned}$$

Where in each equality we're just calling the ith row of $\mathbf{V}$ as $\mathbf{v}_i$. In other words, the new features are just linear combinations of the old features! If we'd like, we could even add biases to the mix, e.g. $\mathbf{a}_i = \mathbf{v}_i^\top \mathbf{x} + b_i$. On the surface, this seems like a pretty powerful idea: let's just take an existing loss function and define it to be over both the original linear model weights $\mathbf{w}$ and the weights of the linear map $\mathbf{V}$. For example, maybe our logistic loss:

$$R(\mathbf{w}) = \sum_{i=1}^{n} \log(1 + \exp(-y_i \mathbf{w}^\top \mathbf{x}_i))$$

becomes:

$$R(\mathbf{w}, \mathbf{V}) = \sum_{i=1}^{n} \log(1 + \exp(-y_i \mathbf{w}^\top \mathbf{V}\mathbf{x}_i))$$

Relatively simple, right? There's only one small problem: we haven't actually done anything! Our predictive model is:

$$h(\mathbf{x}) = \mathbf{w}^\top \mathbf{V}\mathbf{z}$$

But what happens if we group terms like:

$$h(\mathbf{x}) = (\mathbf{w}^\top \mathbf{V})\mathbf{z}$$

Then, for any particular given $\mathbf{w}$, we can just compute $\mathbf{u} = \mathbf{w}^\top \mathbf{V}$ ahead of time, and our predictive model becomes $h(\mathbf{z}) = \mathbf{uz}$. What are the entries of $\mathbf{u}$? Well, $\mathbf{w}^\top \mathbf{V}$ takes $\mathbf{w}$ and computes a dot product with each column of $\mathbf{V}$:

$$u_1 = \mathbf{w}^\top [v_{11}, v_{21}, ..., v_{D1}] = w_1 v_{11} + w_2 v_{21} + \cdots + w_D v_{D1}$$
$$\vdots$$
$$u_d = \mathbf{w}^\top [v_{1d}, v_{2d}, ..., v_{Dd}] = w_1 v_{1d} + w_2 v_{2d} + \cdots + w_D v_{Dd}$$

This is ultimately just a set of d numbers (think about $\mathbf{w}^\top$ as a $1 \times D$ vector and $\mathbf{V}$ as a $D \times d$ matrix, so that $\mathbf{w}^\top \mathbf{V}$ is a $1 \times d$ matrix/vector). Put another way to be painfully obvious, define $\mathbf{w}' = \mathbf{u}^\top$ so that $(\mathbf{w}'^\top)^\top = \mathbf{u}$, then:

$$h(\mathbf{z}) = \mathbf{w}^\top \mathbf{V} \mathbf{z} = (\mathbf{w}^\top \mathbf{V})\mathbf{z} = \mathbf{uz} = \mathbf{w}'^\top \mathbf{z}$$

This is just another linear model with weights $\mathbf{w}'$ instead of $\mathbf{w}$!! In fact, this is really bad news for us: if we use the logistic loss like above, because it's convex, we know we're not just finding **a particular** weight vector $\mathbf{w}$, we know we can find the **best possible** weight vector $\mathbf{w}$. Thus, our feature expansion cannot possibly result in a better model than if we just hadn't done it at all! Uh oh!

2 Neural Networks

Fortunately, the fix is really simple. The key problem we ran into is the fact that $\mathbf{w}^\top \mathbf{V}$ is again a vector, which leads to just another linear model. Anything at all we do to stop us from grouping terms like $\mathbf{w}^\top \mathbf{V} \mathbf{z} = (\mathbf{w}^\top \mathbf{V})\mathbf{z} = \mathbf{w}'^\top \mathbf{z}$ solves the problem. To do this, we introduce a *nonlinear activation function.*

(One Hidden Layer) Neural Networks

Suppose we are given some training set $\mathbf{x}_1, ..., \mathbf{x}_n$ with labels $y_1, ..., y_n$. We think linear models of the form $h(\mathbf{z}) = \mathbf{w}^\top \mathbf{z}$ are too simple. Let's learn a feature map of the form:

$$\phi(\mathbf{x}) = g(\mathbf{V}\mathbf{x})$$

where $g(\cdot)$ **is an *activation function***, and $\mathbf{V} \in \mathbb{R}^{D \times d}$ is a matrix that just maps $\mathbf{x}$ to a higher, $D > d$ dimensional space by taking linear combinations of the existing features. Then, we'll use models of the form:

$$h(\mathbf{z}) = \mathbf{w}^\top \phi(\mathbf{z}) = \mathbf{w}^\top g(\mathbf{V}\mathbf{z})$$

Here, as long as g is any nonlinear function, obviously $\mathbf{w}^\top g(\mathbf{V}\mathbf{z})$ can't be associated in the same way to just get some new $\mathbf{w}'^\top \mathbf{z}$. Typically, g is taken to be a scalar function that we apply elementwise to the product $\mathbf{V}\mathbf{z}$. Here are some commonly used ones:

1. ReLU (Rectified Linear Unit): $g(s) = \max(0, s)$
2. Sigmoid: $g(s) = \frac{1}{1+\exp(-s)}$

3. Tanh: $g(s) = \tanh(s)$

The ReLU in particular is probably the most commonly used activation function these days, for reasons that I can show you very visually in lecture.

Some nomenclature. Time for a vocabulary lesson! Mostly because neural networks are very old, there's some vocabulary surrounding them you need to know. Our final model is:

$$h(\mathbf{z}) = \mathbf{w}^\top g(\mathbf{V}\mathbf{z})$$

Breaking this up even further:

$$\mathbf{a} = g(\mathbf{V}\mathbf{z})$$
$$h(\mathbf{a}) = \mathbf{w}^\top \mathbf{z}$$

We call this a **neural network with one hidden layer**. The mapping $\mathbf{z} \to \mathbf{a}$ via $g(\mathbf{V}\mathbf{z})$ is often called the first hidden layer. The vector produced by this layer $\mathbf{a}$ is often called the **hidden representation** or the **representation after layer 1**. You might even variously see the inputs themselves, $\mathbf{z}$, be referred to as an "input layer".

2.1 Learning

How do we find a good setting of $\mathbf{w}$ and $\mathbf{V}$? Simple: we just plug the above form into whatever loss function we'd like! In this section, suppose as usual we are given a training dataset consisting of features $\mathbf{x}_1, ..., \mathbf{x}_n$ and labels $y_1, ..., y_n$.

Example 1: Classification Say we want to learn a good neural network model for classification. One way to do this would be to start with the logistic loss function again:

$$R(\mathbf{w}) = \sum_{i=1}^{n} \log(1 + \exp(-y_i \mathbf{w}^\top \mathbf{x}_i))$$

And introduce our neural network with one hidden layer instead of just a linear model:

$$R(\mathbf{w}, \mathbf{V}) = \sum_{i=1}^{n} \log(1 + \exp(-y_i \mathbf{w}^\top g(\mathbf{V}\mathbf{x}_i)))$$

The loss above definitely no longer has nice properties like convexity, but for any activation function $g(\cdot)$, you could certainly compute the derivative of R with respect to both $\mathbf{w}$ and the entries of $\mathbf{V}$. Then, we just do the usual gradient descent updates:

$$\mathbf{w}_t \leftarrow \mathbf{w}_{t-1} - \eta \frac{\partial}{\partial \mathbf{w}} R(\mathbf{w}_{t-1}, \mathbf{V}_{t-1})$$
$$\mathbf{V}_t \leftarrow \mathbf{V}_{t-1} - \eta \frac{\partial}{\partial \mathbf{V}} R(\mathbf{w}_{t-1}, \mathbf{V}_{t-1})$$

That's all there is to it for classification!

Example 2: Regression Say we want to learn a good neural network model for regression. One way to do this would be to start with the squared loss function:

$$R(\mathbf{w}) = \sum_{i=1}^{n} (\mathbf{w}^\top \mathbf{x}_i - y_i)^2$$

And plug in our neural network in place of the linear model:

$$R(\mathbf{w}, \mathbf{V}) = \sum_{i=1}^{n} (\mathbf{w}^\top g(\mathbf{V}\mathbf{x}_i) - y_i)^2$$

Of course, there won't be any more convexity or closed form solution here, but that's all we need to do! Now we just apply gradient descent as before.

What about regularization? A lot of the regularizers we've learned about can be used for neural networks. Recall that L2 regularization essentially amounted to the sum of squared weights, $\|\mathbf{w}\|_2^2 = \sum_{i=1}^{d} w_i^2$. We could introduce the following regularizer for a neural net:

$$\text{l2-reg}(\mathbf{w}, \mathbf{V}) = \sum_{i=1}^{D} w_i^2 + \sum_{i=1}^{D} \sum_{j=1}^{d} v_{ij}^2$$

This plays the same intuitive role as it did before: it makes our model less powerful.

3 Neural networks with multiple hidden layers

What if train a neural network and it's still not powerful enough? If you want more power, there are broadly speaking two ways to achieve this:

1. Increase the dimensionality D of your hidden representation. The bigger D gets, the more features your final linear model gets to play with!
2. Stack more layers – do the whole thing over again!

What if, instead of defining a single feature map:

$$\begin{aligned} \mathbf{a} &= g(\mathbf{V}\mathbf{x}) \\ h(\mathbf{a}) &= \mathbf{w}^\top \mathbf{a} \end{aligned}$$

We defined *two* feature maps:

$$\begin{aligned} \mathbf{a}_1 &= g(\mathbf{V}_1\mathbf{x}) \\ \mathbf{a}_2 &= g(\mathbf{V}_2\mathbf{a}_1) \\ h(\mathbf{a}_2) &= \mathbf{w}^\top \mathbf{a}_2 \end{aligned}$$

Here, $\mathbf{x}$ is d dimensional as usual. We map it first to a D_1 dimensional hidden representation $\mathbf{a}_1$ using layer 1. Layer 2 then maps $\mathbf{a}_1$ to a D_2 dimensional hidden representation $\mathbf{a}_2$. We then use a standard linear model using $\mathbf{a}_2$ as features. Our full predictive model becomes:

$$h(\mathbf{z}) = \mathbf{w}^\top g(\mathbf{V}_2 g(\mathbf{V}_1 \mathbf{z}))$$

But it's probably easier to think about the expanded form involving $\mathbf{a}_1, \mathbf{a}_2$ instead. To do learning, we just define our loss function to be over $\mathbf{w}, \mathbf{V}_1, \mathbf{V}_2$ now and use gradient descent. In general, you could imagine repeating this procedure to get neural networks with k hidden layers:

$$\begin{aligned}
\mathbf{a}_1 &= g(\mathbf{V}_1 \mathbf{x}) \\
\mathbf{a}_2 &= g(\mathbf{V}_2 \mathbf{a}_1) \\
&\vdots \\
\mathbf{a}_k &= g(\mathbf{V}_{k+1} \mathbf{a}_k) \\
h(\mathbf{a}_k) &= \mathbf{w}^\top \mathbf{a}_k
\end{aligned}$$

Here, $\mathbf{x}$ has dimension d as usual, and $\mathbf{a}_i$ has dimension D_i.

4 Stochastic Gradient Descent

Modern neural networks have *gigantic* numbers of parameters and sometimes layers. As we gather more and more data and build bigger and bigger models, gradient descent becomes increasingly expensive. The chief solution to this problem is stochastic gradient descent. Recall that standard gradient descent iterates:

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \nabla_{\mathbf{w}} R(\mathbf{w}_{t-1})$$

When $R(\mathbf{w})$ looks like a sum over datapoints, we can move the gradient inside the sum since $\nabla(f + g) = \nabla f + \nabla g$:

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{1}{n} \sum_{i=1}^{n} \nabla_{\mathbf{w}} \ell(h_{\mathbf{w}}(\mathbf{x}_i), y_i)$$

Here, for example, ℓ might be the squared or logistic loss. Computing this sum over all data points can easily exceed any normal amount of RAM or GPU memory. To solve this problem, stochastic gradient descent chooses a *subset of training data points* B, and only computes the derivative over those points:

$$\mathbf{w}_t = \mathbf{w}_{t-1} - \eta \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\mathbf{w}} \ell(h_{\mathbf{w}}(\mathbf{x}_i), y_i)$$

Here, at each iteration we use a different set of data points B_t. The bad news is that this has worse convergence properties. The good news is that, if n grows really large, it becomes our only option!